SPÉCIFICATIONS TECHNIQUES & FONCTIONNELLES

Cahier des Charges — EventPro API

PROJET	SOCLE TECHNIQUE	BASE DE DONNÉES
EventPro Billetterie API	**Java 17 & Spring Boot 3.x (Architecture en couches)**	**PostgreSQL (ACID & Verrous pessimistes)**

> **Principe directeur :** Vendre des billets sans jamais dépasser la capacité disponible. Les réservations bloquent temporairement les places pendant 10 minutes, les paiements sont simulés avec vérification stricte et chaque billet ne peut être composté qu'une seule fois.

1. Présentation du projet

EventPro est une API REST simple et robuste pour gérer des événements et vendre des billets en ligne. Le flux utilisateur est direct : consulter les événements, bloquer des billets pendant 10 minutes, appliquer une réduction facultative, simuler le paiement et générer les billets avec un code unique pour le contrôle à l'entrée.

2. Objectifs fonctionnels et pédagogiques

Appliquer une architecture en couches standard (Controller, Service, Repository, DTO, Mapper, Entity) avec des règles métier claires : gestion du stock de places en base de données et déverrouillage automatique par minuteur des commandes non payées.

3. Technologies imposées

Composant / Technologie	Version / Spécification
Langage & Framework	Java 17, Spring Boot 3.2.x, Spring Web (MVC)
Accès aux données (ORM)	Spring Data JPA, Hibernate ORM 6.4.x
Base de données	PostgreSQL 15+ (Gestion des transactions et des verrous `PESSIMISTIC_WRITE`)
Contrôle des données saisies	Jakarta Bean Validation (Hibernate Validator 8.x) sur DTOs
Conversion Entity / DTO	MapStruct (conversion déclarative performante sans réflexion au runtime)
Construction des objets	Pattern Builder (Lombok `@Builder`) pour des DTO et entités fiables
Monnaie et calculs	Devise en MAD (Dirham marocain), calculs avec `BigDecimal`
Documentation de l'API	Springdoc OpenAPI 2.x / Swagger UI
Outils de test	Apache Maven, JUnit 5, Mockito, Testcontainers (Bonus)
Tâche automatique	Spring Scheduling (`@Scheduled`) pour libérer les places non payées

4. Fonctionnement général

Le parcours est simple et linéaire : 1) Le client choisit un événement publié. 2) Il crée une réservation qui bloque les places pendant 10 minutes. 3) Il saisit éventuellement un code promo. 4) Il confirme son achat via une simulation de paiement. 5) Les billets définitifs sont générés avec un statut valide.

5. Rôles et responsabilités

ADMIN	ORGANIZER	CLIENT	AGENT
Consulte la liste globale des utilisateurs.	Crée, met à jour et publie ses événements.	Consulte le catalogue des événements publiés.	Scanne et composte les billets à l'entrée.
Crée les codes promotionnels généraux.	Configure ses catégories de billets et leurs prix.	Réserve des places (bloquées 10 minutes).	Vérifie si le billet est valide ou déjà scanné.
Supervise l'historique des réservations.	Consulte le total de ses ventes et billets vendus.	Paye sa réservation et télécharge ses billets.	Refuse l'accès si le billet est invalide.
Gère les annulations exceptionnelles.	Annule un événement en cas d'imprévu.	Demande un remboursement en cas d'annulation.	Enregistre la date et l'heure du passage.

6. User Stories (Besoins par acteur)

CLIENT & ACHATS

• **US1 :** Je veux voir la liste des événements et les places restantes.
• **US2 :** Je veux réserver des places bloquées pendant 10 minutes.
• **US3 :** Je veux entrer un code promo pour payer moins cher.
• **US4 :** Je veux payer et obtenir mes billets électroniques.
• **US5 :** Je veux annuler mon achat avant l'événement et être remboursé.

ORGANISATEUR, AGENT & ADMIN

• **US6 :** Je veux créer un événement avec ses tarifs et le publier.
• **US7 :** Je veux voir combien de billets ont été vendus pour mon événement.
• **US8 :** Je veux annuler mon événement si nécessaire avec remboursement auto.
• **US9 :** Je veux scanner les billets et empêcher un second passage.
• **US10 :** Je veux superviser l'ensemble des opérations en tant qu'admin.

7. Règles métier et gestion des places

- **Calcul des places disponibles :** `placesDisponibles = quantiteTotale - (placesReservees + placesVendues)`. Si la quantité demandée dépasse le disponible, la demande est rejetée.
- **Verrouillage pessimiste (PESSIMISTIC_WRITE) :** Lors d'une réservation, un verrou exclusif pessimiste (`PESSIMISTIC_WRITE` via `SELECT ... FOR UPDATE`) bloque la catégorie de billet dans la transaction, prévenant toute survente concurrente.
- **Réservation temporaire (10 minutes) :** La réservation reçoit le statut `PENDING_PAYMENT`. Après 10 minutes sans paiement, elle est expirée et les places redeviennent disponibles pour les autres clients.
- **Confirmation d'achat :** Dès la validation du paiement, les places passent de réservées à vendues, et les billets passent au statut `VALID`.
- **Contrôle d'accès :** Chaque billet est composté une seule fois (passage de `VALID` à `USED`). Tout scan ultérieur retourne une erreur `409 Conflict`.

8. Règles de Remboursement Claires

Le remboursement repose sur une condition chronologique nette calculée par rapport à la date de début de l'événement (`dateDebut`) :

Moment de la demande	Taux remboursé	Impact sur les billets et le stock
48h ou plus avant l'événement (≥ 48h)	**100%** (Remboursement intégral)	Les billets passent à `CANCELLED` et les places redeviennent vendables.
Moins de 48h avant l'événement (< 48h)	**0%** (Refus catégorique)	Aucun remboursement. Les billets restent `VALID` pour le client.
Annulation par l'organisateur	**100%** automatique	Tous les acheteurs sont intégralement remboursés, les billets sont invalidés.

9. Calcul des promotions

Le calcul de la réduction est direct :
• Si *PERCENTAGE* : `Remise = SousTotal × Valeur / 100`.
• Si *FIXED_AMOUNT* : `Remise = Valeur` (sans dépasser le sous-total).
Le montant final à payer est simplement `Total = SousTotal - Remise` (minimum 0 MAD).

10. Entités, Attributs, Méthodes & Relations (Diagramme officiel)

Entité / Classe	Attributs essentiels & Types	Méthodes de domaine	Relations directes
`Utilisateur` *(Abs)*	id (Long), nom (String), email (String)	seConnecter(), seDeconnecter()	Classe mère abstraite
`Administrateur`	Hérite de `Utilisateur`	gererUtilisateurs() : void	Hérite d'Utilisateur
`Organisateur`	Hérite de `Utilisateur`, nomOrganisation (String)	creerEvenement(), modifierEvenement()	1 ➔ 0..* Evenement, Promo
`Client`	Hérite de `Utilisateur`, telephone (String)	rechercherEvenements(), reserverBillets()	1 ➔ 0..* Reservation
`AgentControle`	Hérite de `Utilisateur`	verifierBillet(code : String) : boolean	Hérite d'Utilisateur
`Evenement`	id (Long), titre, description, lieu, dateDebut, dateFin, dateOuvertureVente, dateClotureVente, statut	publier(), annuler(), verifierDisponibilite()	* ➔ 1 Organisateur 1 ➔ 1..* CategorieBillet
`CategorieBillet`	id, nom, prixUnitaire (BigDecimal), quantiteTotale, quotaMaxParClient	verifierStock(quantite : int) : boolean	* ➔ 1 Evenement 1 ➔ 0..* Billet
`Promotion`	id, code, typeReduction, valeurReduction (BigDecimal), dateDebut, dateFin, limiteUtilisation, nombreUtilisations	estValide() : boolean, calculerReduction(m : BigDecimal)	* ➔ 1 Organisateur 0..1 ➔ 0..* Reservation
`Reservation`	id, reference, dateCreation, dateExpiration, statut, montantTotal (BigDecimal)	calculerTotal(), confirmer(), annuler(), expirer()	* ➔ 1 Client, 0..1 Promo 1 ➔ 1..* LigneReservation
`LigneReservation`	id, quantite, prixUnitaire (BigDecimal), sousTotal (BigDecimal)	calculerSousTotal() : BigDecimal	* ➔ 1 Reservation * ➔ 1 CategorieBillet
`Paiement`	id, reference, montant (BigDecimal), datePaiement, statut	payer() : boolean, rembourser(m : BigDecimal)	* ➔ 1 Reservation 1 ➔ 0..* Remboursement
`Remboursement`	id, montant (BigDecimal), motif, dateDemande, statut	demander(), approuver(), refuser()	* ➔ 1 Paiement
`Billet`	id, code, dateEmission, dateControle, statut	valider() : boolean, controler() : boolean	* ➔ 1 LigneReservation * ➔ 1 CategorieBillet

Énumérations associées : • `StatutEvenement` (DRAFT, PUBLISHED, COMPLETED, CANCELLED) | • `TypeReduction` (PERCENTAGE, FIXED_AMOUNT)
• `StatutReservation` (PENDING_PAYMENT, CONFIRMED, EXPIRED, CANCELLED, REFUNDED) | • `StatutPaiement` (PENDING, SUCCESS, FAILED)
• `StatutRemboursement` (PENDING, APPROVED, REJECTED) | • `StatutBillet` (VALID, USED, CANCELLED)

11. Cartographie formelle des associations et multiplicités

HÉRITAGE & GESTION ÉVÉNEMENTIELLE

• **Héritage polymorphique direct :** `Administrateur`, `Organisateur`, `Client` et `AgentControle` étendent la classe mère abstraite `Utilisateur`.
• **Organisation (1 ➔ 0..*) :** Un `Organisateur` gère zéro à plusieurs `Evenement` et crée zéro à plusieurs `Promotion`.
• **Composition tarifaire (1 ➔ 1..*) :** Un `Evenement` propose obligatoirement une à plusieurs `CategorieBillet`.

RÉSERVATION, PAIEMENT & COMPÔTAGE

• **Panier (1 ➔ 1..*) :** Une `Reservation` regroupe une à plusieurs `LigneReservation`, chaque ligne pointant vers une `CategorieBillet`.
• **Règlement & Litiges (1 ➔ 0..*) :** Une réservation est acquittée via `Paiement`, pouvant faire l'objet de demandes de `Remboursement`.
• **Génération unitaire (1 ➔ 0..*) :** Chaque `LigneReservation` confirmée engendre autant d'instances uniques de `Billet` que la quantité achetée.

12. Arborescence complète et Architecture du projet (Package-by-Layer)

```
src/main/java/com/eventpro/
├── controller/ # Endpoints REST (Event, Reservation, Payment, Ticket, CheckIn)
├── service/ # Logique métier & verrous (Event, Reservation, Payment, Ticket, Promo)
├── repository/ # Spring Data JPA (Event, CategorieBillet, Reservation, Paiement, Billet)
├── entity/ # Entités JPA (Utilisateur, Evenement, CategorieBillet, Promotion, Reservation, etc.)
├── dto/ # Request/Response indépendants (assemblés via pattern Builder @Builder)
├── mapper/ # MapStruct (EventMapper, ReservationMapper, PaymentMapper, TicketMapper)
├── scheduler/ # @Scheduled (libération automatique des réservations expirées à 10 min)
├── advice/ # @RestControllerAdvice global retournant ErrorResponse via Builder
└── exception/ # Exceptions métier (InsufficientStockException, ReservationExpired, etc.)
```

13. Objets de transfert de données (DTOs) & Gestion des erreurs HTTP

Données reçues (Request DTO)

- `ReservationRequest` : ID événement, lignes (catégorie + quantité), code promo optionnel.
- `PaymentRequest` : Clé d'idempotence, montant.
- `EventCreateRequest` : Titre, lieu, dates, catégories.

Données renvoyées (Response DTO)

- `ReservationResponse` : Référence, échéance 10 min, statut, montants.
- `TicketResponse` : Code unique, statut d'accès, porteur.
- *Construction :* Implémentation via `@Builder`.

Code HTTP	Nom de l'erreur interne	Dans quelle situation ce code est renvoyé ?
`200 OK / 201`	`SuccessResponse<T>` / DTO nominal	L'opération a réussi (lecture de données ou création d'une ressource).
`400 Bad Request`	`ValidationException`	Informations incorrectes (ex. quantité négative, dates illogiques).
`404 Not Found`	`ResourceNotFoundException`	L'événement, le billet ou la réservation demandée n'existe pas.
`409 Conflict`	`InsufficientStockException`	Il n'y a plus assez de places libres pour valider la réservation.
`409 Conflict`	`TicketAlreadyUsedException`	Le billet présenté a déjà été scanné à l'entrée (statut `USED`).
`409 Conflict`	`IdempotencyConflictException`	La même clé de paiement est réutilisée avec un panier différent.
`409 Conflict`	`ReservationExpiredException`	Le paiement arrive trop tard : les 10 minutes sont déjà écoulées.
`500 Internal`	`SystemException` imprévue	Erreur serveur inattendue. Le message ne doit pas dévoiler le code source.

> **Gestion globale avec @RestControllerAdvice :** Interception centralisée par `GlobalExceptionHandler`. L'objet JSON standard comprend : `timestamp`, `status`, `error`, `message`, `path` et `fieldErrors`.

14. Automatisation & Stratégie de tests

- **Tâche planifiée (@Scheduled) :** Scrute les réservations `PENDING_PAYMENT` dépassant 10 minutes, les passe à `EXPIRED` et restitue les stocks.
- **Tests unitaires & Intégration :** Validation avec Mockito (calculs de prix, remises, remboursements) et Testcontainers PostgreSQL (concurrence sur la dernière place). Lancement via `mvn test`.